LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 5
“REKURSIF”

DISUSUN OLEH:

ITRHOH ANGGUN PAMUNGKAS

109082500117

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

DASAR TEORI

Rekursif adalah teknik dalam pemrograman di mana sebuah fungsi atau prosedur memanggil dirinya sendiri untuk menyelesaikan suatu masalah dengan cara memecahnya menjadi sub-masalah yang lebih kecil. Dalam rekursif terdapat dua komponen utama yaitu base-case sebagai kondisi berhenti dan recursive-case sebagai bagian pemanggilan ulang fungsi itu sendiri. Teknik ini sering digunakan untuk menyelesaikan permasalahan seperti deret Fibonacci, faktorial, dan pencarian nilai pangkat karena lebih sederhana dan terstruktur dibandingkan dengan perulangan biasa. Menurut referensi lain, rekursi juga membantu dalam memodelkan masalah yang memiliki pola berulang secara alami, meskipun perlu diperhatikan agar tidak terjadi rekursi tanpa henti

SOAL LATIHAN

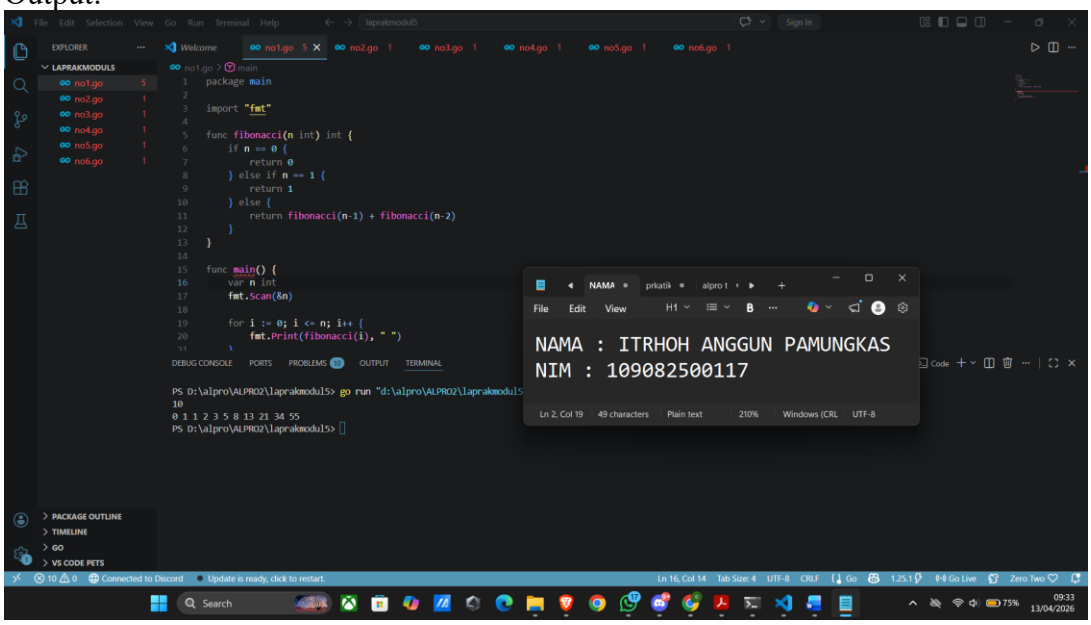
Statement Rekursif

1.

Source Code:

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

Output:



Deskripsi Program:
Program ini membuat fungsi fibonacci(n) yang memanggil dirinya sendiri untuk menghitung nilai deret Fibonacci, dengan kondisi berhenti saat n = 0 atau n = 1. Di dalam main, program akan menampilkan deret dari 0 sampai n dengan cara memanggil fungsi tersebut berulang menggunakan perulangan.

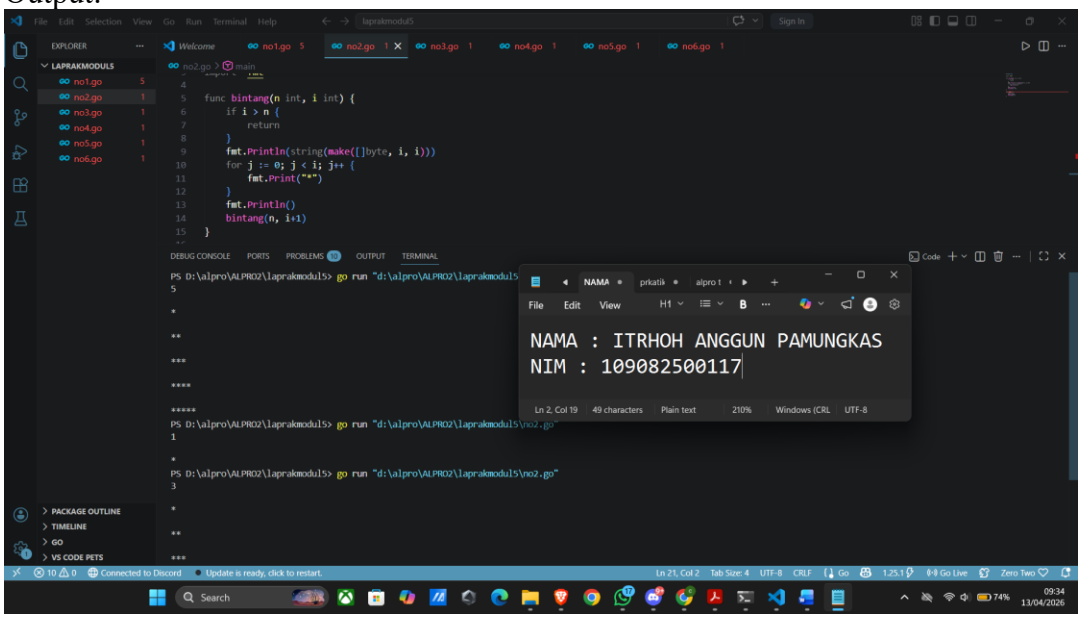
Statement Rekursif

2.

Source Code:

```
package main

import "fmt"

func bintang(n int, i int) {
    if i > n {
        return
    }
    fmt.Println(string(make([]byte, i, i)))
    for j := 0; j < i; j++ {
        fmt.Print("*")
    }
    fmt.Println()
    bintang(n, i+1)
}

func main() {
    var n int
    fmt.Scan(&n)
    bintang(n, 1)
}
```

Output:



Deskripsi Program:
Program ini menggunakan fungsi rekursif bintang(n, i) untuk mencetak bintang dari 1 sampai n secara bertahap. Setiap pemanggilan fungsi akan menambah jumlah bintang dan berhenti saat i lebih dari n.

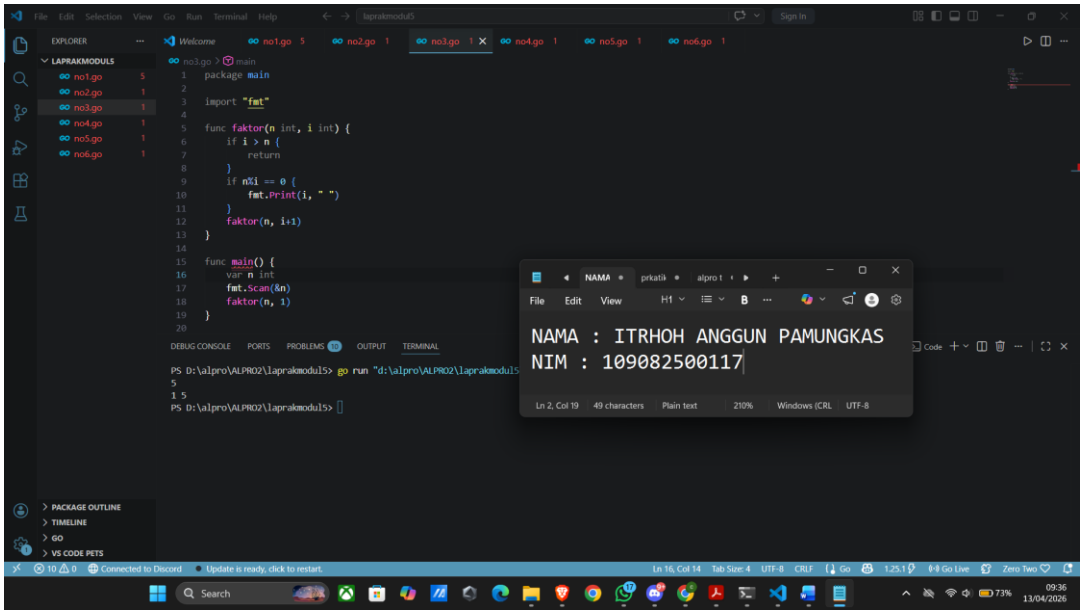
Statement Rekursif

3.

Source Code:

```
package main

import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Scan(&n)
    faktor(n, 1)
}
```

Output:



Deskripsi Program:
Fungsi faktor(n, i) digunakan untuk mengecek apakah i adalah faktor dari n dengan menggunakan operasi modulus. Jika habis dibagi, maka i akan ditampilkan, lalu fungsi memanggil dirinya sendiri dengan nilai i yang bertambah.

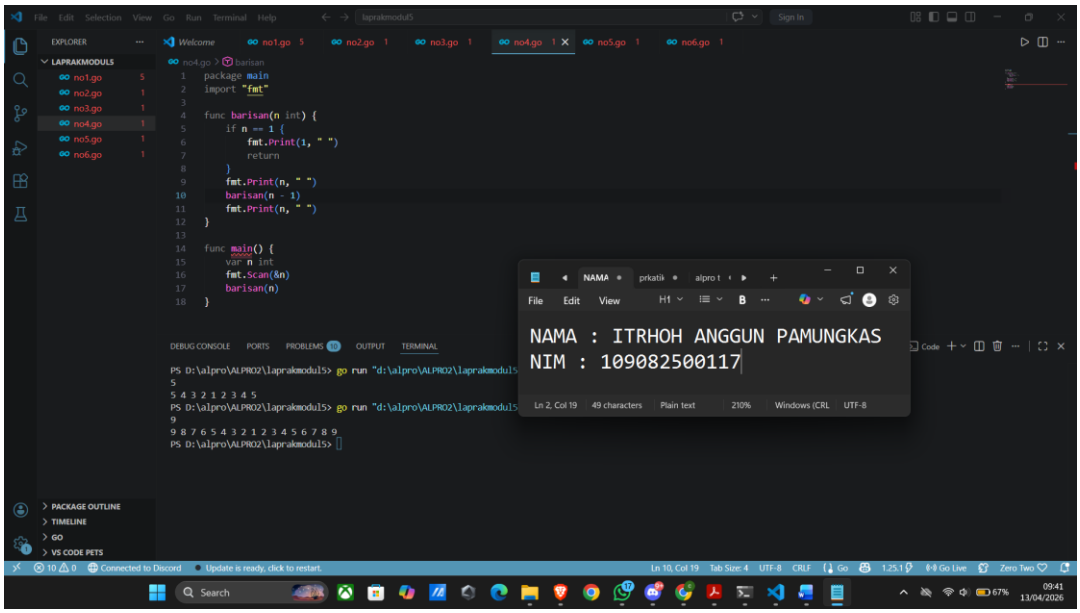
Statement Rekursif

4.

Source Code:

```
package main
import "fmt"

func barisan(n int) {
    if n == 1 {
        fmt.Print(1, " ")
        return
    }
    fmt.Print(n, " ")
    barisan(n - 1)
    fmt.Print(n, " ")
}

func main() {
    var n int
    fmt.Scan(&n)
    barisan(n)
}
```

Output:



Deskripsi Program:
Program ini mencetak angka dari n ke 1 menggunakan rekursif, lalu mencetak kembali dari 1 ke n saat proses kembali (backtracking). Hal ini terjadi karena ada perintah cetak sebelum dan sesudah pemanggilan fungsi rekursif.

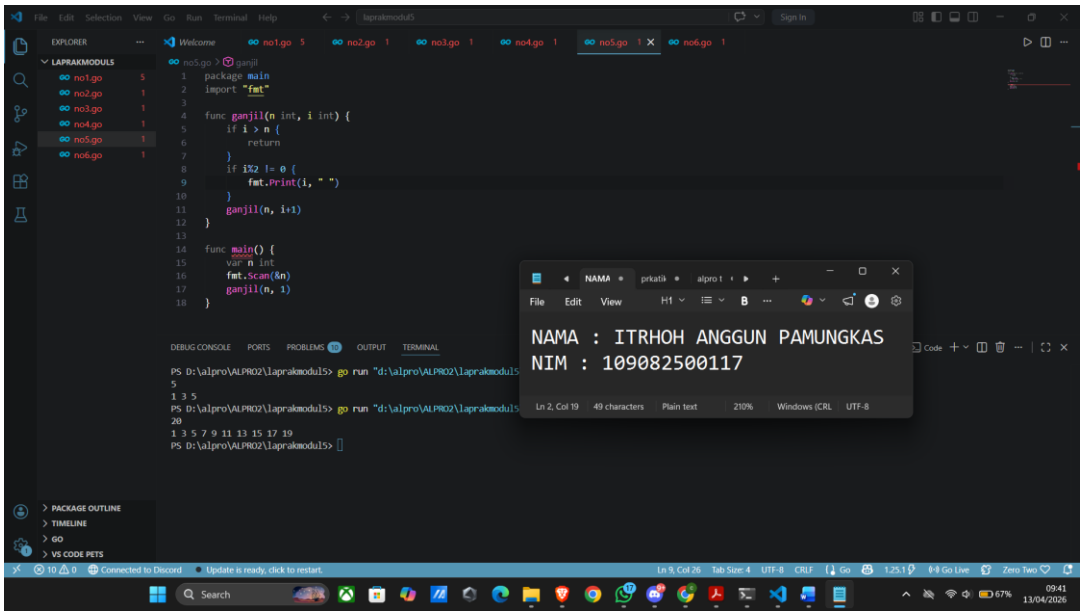
Statement Rekursif

5.

Source Code:

```
package main
import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Print(i, " ")
    }
    ganjil(n, i+1)
}

func main() {
    var n int
    fmt.Scan(&n)
    ganjil(n, 1)
}
```

Output:



Deskripsi Program:
Fungsi ganjil(n, i) akan mengecek setiap angka dari 1 sampai n dan hanya mencetak angka yang habis dibagi 2 dengan sisa. Fungsi ini terus berjalan dengan menambah nilai i hingga melebihi n.

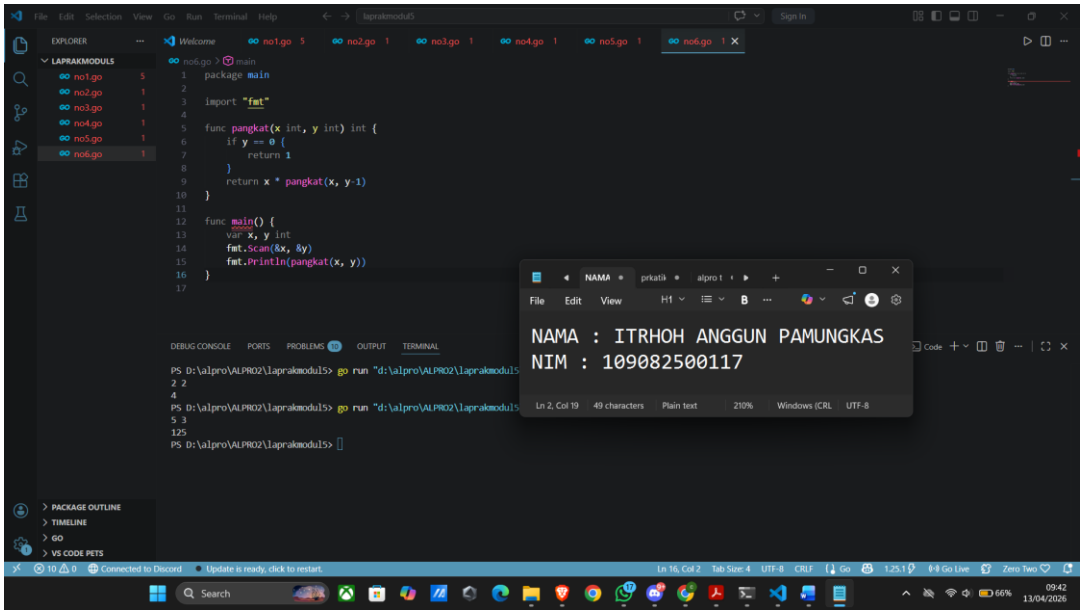
Statement Rekursif

6.

Source Code:

```
package main
import "fmt"

func pangkat(x int, y int) int {
    if y == 0 {
        return 1
    }
    return x * pangkat(x, y-1)
}

func main() {
    var x, y int
    fmt.Scan(&x, &y)
    fmt.Println(pangkat(x, y))
}
```

Output:



Deskripsi Program:
Fungsi pangkat(x, y) menghitung hasil pangkat dengan cara mengalikan x secara berulang menggunakan rekursif. Proses berhenti saat y = 0 karena hasil pangkat berapapun jika pangkat 0 adalah 1.

DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2 – Modul 5

https://www.geeksforgeeks.org/recursion/